



SHOPSPHERE

E-COMMERCE DATABASE

OCT 2026

Complete Project Report

Relational database design • MySQL 8+ • ER modelling • Normalization • SQL analytics



Contribution

Sl. No.	Team Member	Contribution / Responsibility
1	Afifa	Worked on writing and implementing SQL transition queries.
2	Bhuvan	Prepared the complete README and handled the overall project report preparation and documentation.
3	Darsh	Created the database and schema, pushed the project files to GitHub, and worked on testing, debugging, and resolving failures.
4	Fayaz	Designed and prepared the ER (Entity-Relationship) diagrams for the database.
5	Gokul	Worked on database normalization in collaboration with Hemanth.
6	Hemanth	Worked on normalization and performed testing and verification of all SQL queries.



Project Components

Database	MySQL 8+
Tables	11
Total Records	202
Business Analysis Areas	10
Normalization	3NF-oriented relational schema



Table of Contents

Section	Topic
1	Project Overview
2	Objectives and Technology
3	Database Architecture
4	Entity and Table Documentation
5	Relationships and Keys
6	Normalization and Integrity
7	Dataset Summary
8	Business Questions and Analytics
9	Validation and Quality Checks
10	Project Strengths
11	Future Enhancements
12	Conclusion

Implementation note

The ShopSphere database is implemented using MySQL 8+. The project includes the database schema, sample data, analytical SQL queries, and validation scripts required to create and analyze the database. While some database images or generic references may mention PostgreSQL, the actual supplied SQL uses MySQL-specific constructs such as **USE shopsphere** and **AUTO_INCREMENT**, and all business queries in this report are written and tested on MySQL 8+.



1. Project Overview

ShopSphere is a relational database for an e-commerce platform. It models the major stages of the order lifecycle: customer management, addresses, product catalogue, categories, inventory, orders, order items, payments, shipments, reviews and suppliers.

The updated dataset contains **202 sample rows** distributed across **11 tables**. The project also includes analytical SQL for business questions and a dedicated validation script for duplicate and referential-integrity checks.

Core capabilities

- Store customer and address information.
- Organize products using categories and parent categories.
- Track product prices and inventory.
- Record customer orders and individual order items.
- Track payments and their status.
- Track shipments and delivery dates.
- Store and analyze product reviews.
- Represent product-supplier many-to-many relationships.
- Answer practical business questions with SQL.
- Validate normalization and referential integrity.

Dataset summary

Table	Sample Rows
Customers	14
Addresses	16
Categories	7
Products	16
Suppliers	6
Product Suppliers	21
Orders	20
Order Items	32
Payments	20
Shipments	20
Reviews	30
TOTAL	202



2. Objectives and Technology

Objectives

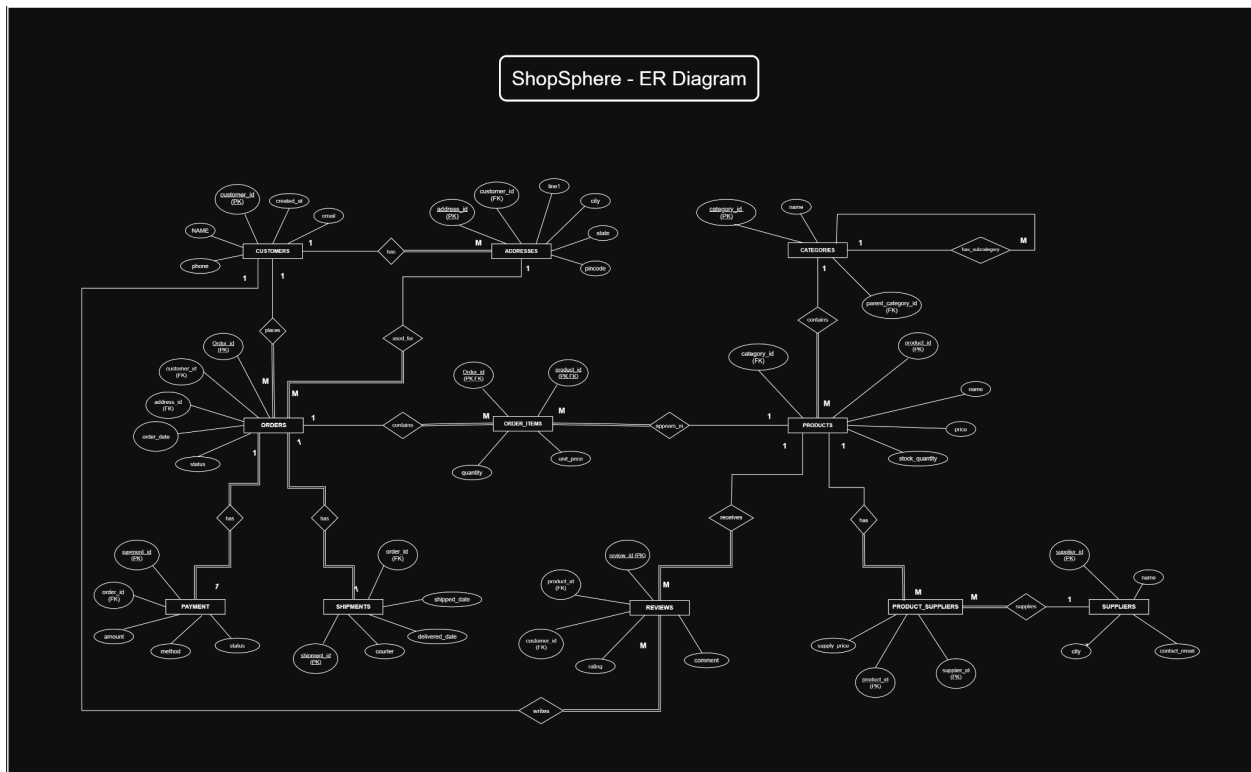
- Design a structured relational database for an e-commerce platform.
- Minimize redundancy through normalized entity separation.
- Enforce data consistency with primary keys, foreign keys and constraints.
- Provide a realistic sample dataset for analysis.
- Demonstrate practical SQL joins and aggregate queries.
- Validate the resulting database for common integrity problems.

Technology and concepts

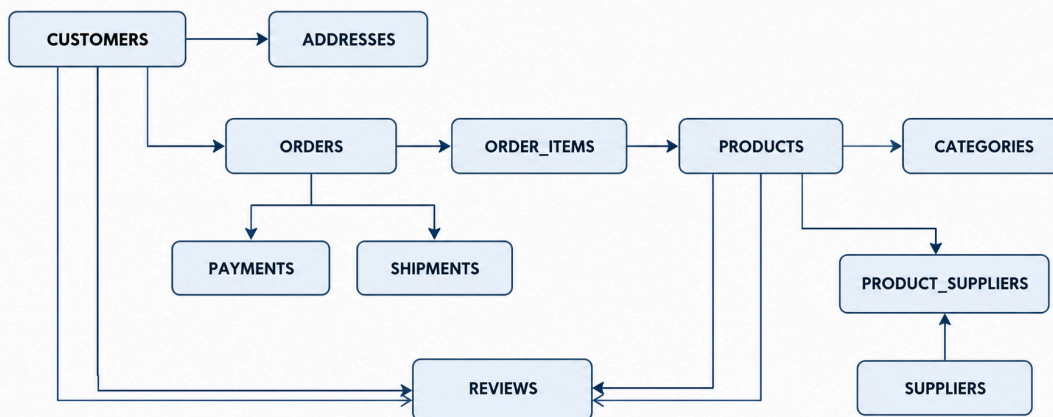
Component	Implementation
Database	MySQL 8+
Model	Relational
Query Language	SQL
Schema size	11 tables
Sample data	202 rows
Normalization	3NF-oriented relational design
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK
Analytics	JOIN, GROUP BY, HAVING, ORDER BY, SUM, AVG, COUNT, ROUND

3. Database Architecture

The architecture separates the core business entities into independent tables and connects them using foreign keys. Many-to-many relationships are resolved through junction tables.



High-level data flow





4. Entity and Table Documentation

Table	Main Attributes	Purpose	Rows
CUSTOMERS	customer_id, name, email, phone, created_at	Customer accounts and contact information.	14
ADDRESSES	address_id, customer_id, line1, city, state, pincode	Addresses belonging to customers.	16
CATEGORIES	category_id, name, parent_category_id	Product categories and hierarchy.	7
PRODUCTS	product_id, name, category_id, price, stock_qty	Catalogue products, prices and inventory.	16
SUPPLIERS	supplier_id, name, contact_email, city	Supplier information.	6
PRODUCT_SUPPLIERS	product_id, supplier_id, supply_price	Product–supplier junction table (many-to-many).	21
ORDERS	order_id, customer_id, address_id, order_date, status	Customer orders and current status.	20
ORDER_ITEMS	order_id, product_id, quantity, unit_price	Products contained in each order.	32
PAYMENTS	payment_id, order_id, amount, method, status	Payment information.	20
SHIPMENTS	shipment_id, order_id, courier, shipped_date, delivered_date	Shipment and delivery information.	20
REVIEWS	review_id, customer_id, product_id, rating, comment	Customer ratings and comments.	30

Important constraints

- Customers: customer_id is the primary key and email is unique.
- Products: price and stock_qty cannot be negative.
- Order items: quantity must be positive and unit_price cannot be negative.
- Payments: amount cannot be negative and status is restricted to defined values.
- Reviews: rating must be between 1 and 5; a customer can review a product only once.
- Junction tables use composite primary keys to prevent duplicate relationships.



5. Relationships and Keys

Relationship summary

Relationship	Cardinality	Purpose
Customers → Addresses	1 : M	A customer may have multiple addresses.
Customers → Orders	1 : M	A customer may place multiple orders.
Addresses → Orders	1 : M	An address may be used by multiple orders.
Categories → Products	1 : M	A category can contain multiple products.
Categories → Categories	1 : M	Parent categories can contain child categories.
Orders → Order Items	1 : M	An order contains multiple line items.
Products → Order Items	1 : M	A product can occur in many order lines.
Orders → Payments	1 : 1	One payment record per order in this design.
Orders → Shipments	1 : 1	One shipment record per order in this design.
Customers → Reviews	1 : M	A customer may write multiple reviews.
Products → Reviews	1 : M	A product may receive multiple reviews.
Products ↔ Suppliers	M : N	Resolved using product_suppliers.

Primary keys

Table	Primary Key
Customers	customer_id
Addresses	address_id
Categories	category_id
Products	product_id
Suppliers	supplier_id
Orders	order_id
Order Items	(order_id, product_id)
Payments	payment_id
Shipments	shipment_id
Reviews	review_id
Product Suppliers	(product_id, supplier_id)

6. Normalization and Integrity

Approach

We started from a single unnormalised "orders" sheet, identified the functional dependencies (FDs), and decomposed it until every table was in Third Normal Form (3NF). A table is in 3NF when every non-key attribute depends on the key, the whole key, and nothing but the key.

Unnormalised design (before)

order_id	order_date	customer_name	customer_email	delivery_address	products	quantities	prices	payment_method	courier
101	2026-09-01	Aarav Sharma	aarav@mail.com	12 MG Road, Bengaluru 560001	Headphones, T-Shirt	1, 2	2999, 499	UPI	BlueDart
102	2026-09-03	Aarav Sharma	aarav@mail.com	12 MG Road, Bengaluru 560001	Running Shoes	1	1999	Card	Delhivery

Problems (anomalies):

- **Not atomic (violates 1NF):** the products, quantities and prices columns hold several values in one cell.
- **Update anomaly:** if Aarav changes his email, every one of his order rows must be edited. Missing one leaves inconsistent data.
- **Insertion anomaly:** a new customer or product cannot be stored until an order exists for it.
- **Deletion anomaly:** deleting the only order for a product also deletes the product's name and price.
- **Redundancy:** customer and address details are repeated on every order.

Decomposition (after)

Step 1 – 1NF: Repeating groups were moved into their own rows. Each product on an order became one row in order_items (order_id, product_id, quantity, unit_price). Every column now holds a single value, and every table has a primary key.

Step 2 – 2NF: In order_items the key is (order_id, product_id). quantity and unit_price depend on the *whole* key, since they describe one product on one order. Product name and current price depend only on product_id, so they

moved to products. Likewise, order date and status depend only on order_id and stay in orders. No partial dependencies remain.

Step 3 – 3NF: Customer details (name, email, phone) depend on customer_id, not on order_id, so they moved to customers. Address details moved to addresses. Payment and shipment details were split into payments and shipments. Category and supplier data got their own tables, with product_suppliers resolving the many-to-many link. No non-key attribute depends on another non-key attribute.

Result: 11 tables: customers, addresses, categories, products, orders, order_items, payments, shipments, reviews, suppliers, product_suppliers.

Functional dependencies and normal form per table

Table	Primary key	Functional dependencies	Highest NF
customers	customer_id	customer_id → name, email, phone, created_at; email → customer_id (UNIQUE)	3NF
addresses	address_id	address_id → customer_id, line1, city, state, pincode	3NF (see note 1)
categories	category_id	category_id → name, parent_category_id; name → category_id (UNIQUE)	3NF
products	product_id	product_id → name, category_id, price, stock_qty	3NF
orders	order_id	order_id → customer_id, address_id, order_date, status	3NF (see note 2)
order_items	(order_id, product_id)	(order_id, product_id) → quantity, unit_price	3NF (see note 3)
payments	payment_id	payment_id → order_id, amount, method, status; order_id → payment_id (UNIQUE)	3NF
shipments	shipment_id	shipment_id → order_id, courier, shipped_date, delivered_date; order_id → shipment_id (UNIQUE)	3NF
reviews	review_id	review_id → customer_id, product_id, rating, comment; (customer_id, product_id) → review_id (UNIQUE)	3NF

suppliers	supplier_id	supplier_id → name, contact_email, city; contact_email → supplier_id (UNIQUE)	3NF
product_suppliers	(product_id, supplier_id)	(product_id, supplier_id) → supply_price	3NF

In every table, each determinant is a key (primary or candidate), so the tables are also in BCNF.

Design decisions related to normalization

1. **Pincode in addresses.** In theory pincode → city, state, which would be a transitive dependency. We did not create a separate pincode table because one pincode can cover several localities and sometimes more than one city name, so it is not a strict dependency in real data. A lookup table would add joins without removing real redundancy.
2. **customer_id in orders.** Since address_id → customer_id, the customer could be derived through the address. We keep it on purpose and enforce a composite foreign key (address_id, customer_id) → addresses. This guarantees that an order's delivery address belongs to the customer who placed it, so the repeated value can never become inconsistent.
3. **unit_price in order_items.** It is a copy of the price at purchase time. products.price is the current price and can change, but the order line must keep what the customer actually paid. It depends on the whole key (order_id, product_id), so 2NF and 3NF are not violated.
4. **No stored order total.** A stored total would be derived data that can drift from order_items. Totals come from the view v_order_totals, and the view v_payment_mismatches checks that each payment equals its order total.

Normalization validation

Normalization.sql runs validation queries (duplicate order items, invalid customer references, address-customer mismatches, orphan records). All of them return zero rows, which confirms the design behaves as normalised in practice.



7. Dataset Summary

Table	Rows
Customers	14
Addresses	16
Categories	7
Products	16
Suppliers	6
Product Suppliers	21
Orders	20
Order Items	32
Payments	20
Shipments	20
Reviews	30
TOTAL	202

Sample domain coverage

- Categories include Electronics, Fashion, Home & Kitchen and Books, with Mobiles, Laptops and Audio as child categories under Electronics.
- Products include phones, laptops, audio devices, fashion products, home appliances and books.
- The revised dataset contains 20 orders, including Delivered, Shipped, Confirmed, Pending and Cancelled statuses.
- Payment methods include UPI, Card and Net Banking; the revised dataset contains 20 payment records.
- Shipment records cover 20 orders and include courier, shipped date and optional delivered date.
- Reviews now contain 30 records and use a 1-5 rating scale with customer comments.



8. Business Questions and Analytics

The project contains ten business questions. The supplied assumptions state that cancelled orders are excluded from sales/revenue calculations, only Paid payments count toward customer value, low inventory means `stock_qty < 10`, and a product needs at least two reviews to appear in the highest-rated-products query.

Q1 — Best-selling products

Uses `SUM(quantity)` across `order_items`, joins orders to exclude cancelled orders, then sorts by total units sold.

Product	Units Sold
Wireless Headphones	4
Mens T-Shirt	4
Bluetooth Speaker	3
Running Shoes	3
Python Programming Book	3
Samsung Galaxy S24	2
Data Science Handbook	2

Q2 — Most valuable customers

Ranks customers by total amount paid on non-cancelled orders, considering only payment records with status = Paid.

Customer	Total Paid
Aarav Sharma	₹140,494
Riya Nair	₹74,096
Arjun Kumar	₹67,998
Rahul Verma	₹58,999
Karan Mehta	₹55,696

Q3 — Revenue by category

Revenue is calculated as quantity × unit_price for non-cancelled orders.

Category	Revenue
Mobiles	₹309,994
Laptops	₹176,997
Audio	₹22,991
Fashion	₹17,788
Home & Kitchen	₹9,498
Books	₹5,994

Q4 — Average order value

The query first calculates each non-cancelled order total and then averages those order totals.

Average order value: ₹30,181.22

Q5 — Low inventory

The low-inventory threshold is stock_qty < 10. Products meeting the condition are listed below.

Product	Current Stock (stock_qty)
iPhone 15	7
Dell Inspiron Laptop	5
HP Pavilion Laptop	3
Lenovo IdeaPad Slim	7
Smart Watch	6
Coffee Maker	4
Air Fryer	9

Q6 — Highest-rated products

Product	Average Rating	Reviews
Samsung Galaxy S24	4.67	3
Wireless Headphones	4.67	3
Running Shoes	3.50	2

Q7 — Customers with no orders

A LEFT JOIN identifies customers whose order_id is NULL. The sample identifies Dev Malhotra and Nisha Menon.

Q8 — Orders by status

Status	Order Count
Delivered	10
Shipped	4
Confirmed	2
Pending	2
Cancelled	2

Q9 — Orders awaiting payment

Order	Customer	Amount	Method
5	Ananya Rao	₹5,097	Card
16	Vikram Singh	₹1,499	Card

Q10 — Supplier/product relationships

The query joins suppliers, product_suppliers and products to report supplier, product and supply price. This demonstrates practical use of a many-to-many relationship.

OUTPUT SUMMARY after executing create_tables.sql and insert_tables.sql.
The following tables were successfully generated within the “ShopSphere” schema.

Table: addresses Columns: <u>address_id</u> int AI PK customer_id int line1 varchar(200) city varchar(100) state varchar(100) pincode varchar(10)	Table: categories Columns: <u>category_id</u> int AI PK name varchar(100) parent_category_id int	Table: customers Columns: <u>customer_id</u> int AI PK name varchar(100) email varchar(100) phone varchar(15) created_at datetime	Table: order_items Columns: <u>order_id</u> int PK <u>product_id</u> int PK quantity int unit_price decimal(10,2)	Table: orders Columns: <u>order_id</u> int AI PK customer_id int address_id int order_date datetime status varchar(30)
Table: payments Columns: <u>payment_id</u> int AI PK order_id int amount decimal(10,2) method varchar(30) status varchar(30)	Table: product_suppliers Columns: <u>product_id</u> int PK <u>supplier_id</u> int PK supply_price decimal(10,2)	Table: products Columns: <u>product_id</u> int AI PK name varchar(150) category_id int price decimal(10,2) stock_qty int	Table: reviews Columns: <u>review_id</u> int AI PK customer_id int product_id int rating int comment varchar(500)	Table: shipments Columns: <u>shipment_id</u> int AI PK order_id int courier varchar(100) shipped_date date delivered_date date
	Table: suppliers Columns: <u>supplier_id</u> int AI PK name varchar(150) contact_email varchar(100) city varchar(100)			

SCHEMAS

Filter objects

mydatabase

sakila

shopisphere

Tables

addresses

categories

customers

order_items

orders

payments

product_suppliers

products

reviews

shipments

suppliers

Views

Stored Procedures

Functions

sys

world

Administration

Schemas

Information

Table: customers

Columns:

customer_id int AI PK

name varchar(100)

email varchar(100)

phone varchar(15)

created_at datetime

Limit to 1000 rows

Q1: What are the best-selling products?

SELECT
p.product_id,
p.name,
SUM(oi.quantity) AS total_units_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id

Result Grid

Filter Rows

Export

Wrap Cell Content

product_id	name	total_units_sold
7	Wireless Headphones	4
11	Mens T-Shirt	4
8	Bluetooth Speaker	3
10	Running Shoes	3
15	Python Programming Book	3
1	Samsung Galaxy S24	2
16	Data Science Handbook	2
2	iPhone 15	1
3	OnePlus Nord CE 4	1
14	Full Immersion 1 earbuds	1

Result 2 x

Output

#	Time	Action	Message	Duration / Fetch
36	22:53:10	SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers UNION ALL SELECT 'addresses' AS table_name, COUNT(*) AS row_count FROM addresses	11 row(s) returned	0.000 sec / 0.000 sec
37	22:53:11	SELECT ((SELECT COUNT(*) FROM customers) + (SELECT COUNT(*) FROM addresses))	1 row(s) returned	0.000 sec / 0.000 sec
38	22:53:11	SELECT 150 AS target_rows, ((SELECT COUNT(*) FROM customers) + (SELECT COUNT(*) FROM addresses))	1 row(s) returned	0.000 sec / 0.000 sec
39	22:53:52	SELECT * FROM shopisphere.customers LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
40	23:29:50	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.supply_price AS unit_price, ps.quantity AS quantity FROM product_suppliers ps JOIN products p ON ps.product_id = p.product_id	10 row(s) returned	0.000 sec / 0.000 sec
41	23:31:13	SELECT p.product_id, p.name, SUM(oi.quantity) AS total_units_sold FROM products p JOIN order_items oi ON p.product_id = oi.product_id	15 row(s) returned	0.000 sec / 0.000 sec

Navigator

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

27
28 -- Q2. Who are the most valuable customers?
29 -- Ranked by total amount paid for non-cancelled orders.
30 *
31 SELECT
32     c.customer_id,
33     c.name,
34     SUM(p.amount) AS total_spent
35 FROM customers c
36 JOIN orders o ON c.customer_id = o.customer_id
37 JOIN order_items oi ON o.order_id = oi.order_id
38 JOIN product_supplies ps ON oi.product_id = ps.product_id
39 WHERE o.status != 'cancelled'
40 
```

Result Grid

customer_id	name	total_spent
1	Aarav Sharma	140494.00
2	Riya Nar	74096.00
5	Arjun Kumar	67998.00
9	Rahul Verma	58999.00
3	Karan Mehta	55696.00
7	Vikram Singh	28997.00
10	Ishita Kapoor	7997.00
6	Heena Iyer	7498.00
8	Gehna Patel	2398.00

Table: customers

Columns: customer_id, name, email, phone, created_at

Information

Table: customers

Columns: customer_id, name, email, phone, created_at

Object Info Session

Output

#	Time	Action	Message	Duration / Fetch
37	22:53:11	SELECT ((SELECT COUNT(*) FROM customers) * (SELECT COUNT(*) FROM addresses) * ...	1 row(s) returned	0.000 sec / 0.000 sec
38	22:53:11	SELECT 150 AS target_rows, ((SELECT COUNT(*) FROM customers) * (SELECT COUNT(*) ...	1 row(s) returned	0.000 sec / 0.000 sec
39	22:53:52	SELECT * FROM shoppere.customers LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
40	23:29:50	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 row(s) returned	0.000 sec / 0.000 sec
41	23:31:13	SELECT p.product_id, p.name, SUM(oi.quantity) AS total_units_sold FROM products p JOIN order...	15 row(s) returned	0.000 sec / 0.000 sec
42	23:31:40	SELECT c.customer_id, c.name, SUM(p.amount) AS total_spent FROM customers c JOIN orders o ON...	9 row(s) returned	0.000 sec / 0.000 sec

Navigator

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

42
43
44 -- Q3. What is the revenue by category?
45 *
46 SELECT
47     c.category_id,
48     c.name AS category_name,
49     SUM(oi.quantity * oi.unit_price) AS revenue
50 FROM categories c
51 JOIN products p ON c.category_id = p.category_id
52 JOIN order_items oi ON p.product_id = oi.product_id
53 
```

Result Grid

category_id	category_name	revenue
5	Mobiles	224996.00
6	Laptops	176997.00
7	Audio	17993.00
2	Fashion	12192.00
3	Home & Kitchen	9498.00
4	Books	5095.00

Table: customers

Columns: customer_id, name, email, phone, created_at

Information

Table: customers

Columns: customer_id, name, email, phone, created_at

Object Info Session

Output

#	Time	Action	Message	Duration / Fetch
38	22:53:11	SELECT 150 AS target_rows, ((SELECT COUNT(*) FROM customers) * (SELECT COUNT(*) ...	1 row(s) returned	0.000 sec / 0.000 sec
39	22:53:52	SELECT * FROM shoppere.customers LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
40	23:29:50	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 row(s) returned	0.000 sec / 0.000 sec
41	23:31:13	SELECT p.product_id, p.name, SUM(oi.quantity) AS total_units_sold FROM products p JOIN order...	15 row(s) returned	0.000 sec / 0.000 sec
42	23:31:40	SELECT c.customer_id, c.name, SUM(p.amount) AS total_spent FROM customers c JOIN orders o ON...	9 row(s) returned	0.000 sec / 0.000 sec
43	23:32:06	SELECT c.category_id, c.name AS category_name, SUM(oi.quantity * oi.unit_price) AS revenue FROM...	6 row(s) returned	0.000 sec / 0.000 sec

Navigator

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

58
59 -- Q4. What is the average order value?
60 *
61 SELECT
62     ROUND(AVG(order_total), 2) AS average_order_value
63 FROM (
64     SELECT
65         o.order_id,
66         SUM(oi.quantity * oi.unit_price) AS order_total
67     FROM orders o
68     JOIN order_items oi ON o.order_id = oi.order_id
69 )
70 
```

Result Grid

average_order_value
34367.00

Table: customers

Columns: customer_id, name, email, phone, created_at

Information

Table: customers

Columns: customer_id, name, email, phone, created_at

Object Info Session

Output

#	Time	Action	Message	Duration / Fetch
39	22:53:52	SELECT * FROM shoppere.customers LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
40	23:29:50	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 row(s) returned	0.000 sec / 0.000 sec
41	23:31:13	SELECT p.product_id, p.name, SUM(oi.quantity) AS total_units_sold FROM products p JOIN order...	15 row(s) returned	0.000 sec / 0.000 sec
42	23:31:40	SELECT c.customer_id, c.name, SUM(p.amount) AS total_spent FROM customers c JOIN orders o ON...	9 row(s) returned	0.000 sec / 0.000 sec
43	23:32:06	SELECT c.category_id, c.name AS category_name, SUM(oi.quantity * oi.unit_price) AS revenue FROM...	6 row(s) returned	0.000 sec / 0.000 sec
44	23:32:27	SELECT ROUND(AVG(order_total), 2) AS average_order_value FROM (SELECT o.order_id, S...	1 row(s) returned	0.016 sec / 0.000 sec

Navigator

SCHEMAS

- Filter objects
- mydatabase
 - sales
 - shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_supplies
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Table: customers

Columns:

- customer_id int AI PK
- name varchar(10)
- email varchar(10)
- phone varchar(10)
- created_at datetime

Object Info Session

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

68 GROUP BY o.order_id
69 ) AS order_totals;
70
71 -- Q5. Which products have low inventory?
72 -- Low inventory threshold = fewer than 10 units.
73
74 SELECT
75   product_id,
76   name,
77   stock_qty
78 FROM products
79 WHERE stock_qty < 10
80 ORDER BY stock_qty;

```

Result Grid

product_id	name	stock_qty
5	HP Pavilion Laptop	3
13	Coffee Maker	4
4	Dell Inspiron Laptop	5
9	Smart Watch	6
6	Lenovo IdeaPad Slim	7
2	iPhone 15	8
14	Air Fryer	9

products 6 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
40	23:29:50	SELECT s.supplier_name, s.name AS supplier_name, p.product_id, p.name AS product_name, p.stock_qty	15 rows returned	0.000 sec / 0.000 sec
41	23:31:10	SELECT p.product_id, p.name, SUM(Mp.quantity) AS total_units_sold FROM products p JOIN order_items o ON p.product_id = o.product_id	9 rows returned	0.000 sec / 0.000 sec
42	23:31:40	SELECT c.customer_id, c.name, SUM(Mp.amount) AS total_spent FROM customers c JOIN orders o ON c.customer_id = o.customer_id	6 rows returned	0.000 sec / 0.000 sec
43	23:32:06	SELECT c.category_id, c.name AS category_name, SUM(Mp.quantity * o.unit_price) AS revenue FROM categories c JOIN product_supplies p ON c.category_id = p.category_id	6 rows returned	0.000 sec / 0.000 sec
44	23:32:27	SELECT ROUND(AVG(order_total), 2) AS average_order_value FROM (SELECT o.order_id, S...	1 row returned	0.016 sec / 0.000 sec
45	23:32:59	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty	7 rows returned	0.000 sec / 0.000 sec

Navigator

SCHEMAS

- Filter objects
- mydatabase
 - sales
 - shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_supplies
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Table: customers

Columns:

- customer_id int AI PK
- name varchar(10)
- email varchar(10)
- phone varchar(10)
- created_at datetime

Object Info Session

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

81
82
83 -- Q6. Which products have the highest ratings?
84 -- Only products with at least 2 reviews are considered.
85
86 SELECT
87   p.product_id,
88   p.name,
89   ROUND(AVG(r.rating), 2) AS average_rating,
90   COUNT(r.review_id) AS review_count
91 FROM products p
92 JOIN reviews r ON p.product_id = r.product_id
93 WHERE review_count >= 2
94 ORDER BY average_rating DESC;

```

Result Grid

product_id	name	average_rating	review_count
1	Samsung Galaxy S24	4.67	3
7	Wireless headphones	4.67	3
10	Running Shoes	3.50	2

Result 7 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
41	23:31:10	SELECT p.product_id, p.name, SUM(Mp.quantity) AS total_units_sold FROM products p JOIN order_items o ON p.product_id = o.product_id	15 rows returned	0.000 sec / 0.000 sec
42	23:31:40	SELECT c.customer_id, c.name, SUM(Mp.amount) AS total_spent FROM customers c JOIN orders o ON c.customer_id = o.customer_id	9 rows returned	0.000 sec / 0.000 sec
43	23:32:06	SELECT c.category_id, c.name AS category_name, SUM(Mp.quantity * o.unit_price) AS revenue FROM categories c JOIN product_supplies p ON c.category_id = p.category_id	6 rows returned	0.000 sec / 0.000 sec
44	23:32:27	SELECT ROUND(AVG(order_total), 2) AS average_order_value FROM (SELECT o.order_id, S...	1 row returned	0.016 sec / 0.000 sec
45	23:32:59	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty	7 rows returned	0.000 sec / 0.000 sec
46	23:33:21	SELECT p.product_id, p.name, ROUND(AVG(r.rating), 2) AS average_rating, COUNT(r.review_id) AS review_count	3 rows returned	0.016 sec / 0.000 sec

Navigator

SCHEMAS

- Filter objects
- mydatabase
 - sales
 - shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_supplies
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Table: customers

Columns:

- customer_id int AI PK
- name varchar(10)
- email varchar(10)
- phone varchar(10)
- created_at datetime

Object Info Session

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

```

97
98
99 -- Q7. Which customers have never placed an order?
100
101 SELECT
102   c.customer_id,
103   c.name,
104   c.email
105 FROM customers c
106 LEFT JOIN orders o ON c.customer_id = o.customer_id
107 WHERE o.customer_id IS NULL;

```

Result Grid

customer_id	name	email
11	Dev Mahotra	dev.mahotra@gmail.com
12	Naha Menon	naha.menon@gmail.com

Result 8 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
44	23:32:27	SELECT ROUND(AVG(order_total), 2) AS average_order_value FROM (SELECT o.order_id, S...	1 row returned	0.016 sec / 0.000 sec
45	23:32:59	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty	7 rows returned	0.000 sec / 0.000 sec
46	23:33:21	SELECT p.product_id, p.name, ROUND(AVG(r.rating), 2) AS average_rating, COUNT(r.review_id) AS review_count	3 rows returned	0.016 sec / 0.000 sec
47	23:33:42	-Q7 Which customers have never placed an order? SELECT c.customer_id, c.name, c.email FROM c...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.016 sec
48	23:33:46	-Q7 Which customers have never placed an order? SELECT c.customer_id, c.name, c.email FROM c...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
49	23:35:21	SELECT c.customer_id, c.name, c.email FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id	2 rows returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- mydatabase
- sakila
- shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_suppliers
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Administration Schemas Information

No object selected

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

112 -- Additional Business Questions

113

114

115 -- Q8. How many orders are there for each order status?

116

117 SELECT

118 status,

119 COUNT(*) AS order_count

120 FROM orders

121 GROUP BY status

Result Grid

supplier_id	supplier_name	product_id	product_name	supply_price
1	TechSource India	4	Dell Inspiron Laptop	5700.00
1	TechSource India	5	HP Pavilion Laptop	5100.00
1	TechSource India	6	Lenovo Ideapad Slim	4500.00
1	TechSource India	9	Smart Watch	3300.00
2	Mobitel Distributors	1	Samsung Galaxy S24	6200.00
2	Mobitel Distributors	2	iPhone 15	5300.00
2	Mobitel Distributors	3	OnePlus Nord CE 4	2150.00
4	FashionKart Wholesale	10	Running Shoes	1800.00
6	AudioMax Suppliers	7	Wireless Headphones	2200.00
6	AudioMax Suppliers	8	Bluetooth Speaker	1400.00

Result 11 Result 12 Result 13 Result 14 products 15 Result 16 Result 17 Result 18 Result 19 Result 20 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
19	22:44:02	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty ...	7 rows returned	0.000 sec / 0.000 sec
20	22:44:02	SELECT p.product_id, p.name, ROUND(AVG(r.rating), 2) AS average_rating, COUNT(r.review_id) A...	3 rows returned	0.000 sec / 0.000 sec
21	22:44:02	SELECT c.customer_id, c.name, c.email FROM customers c LEFT JOIN orders o ON c.customer_id ...	2 rows returned	0.000 sec / 0.000 sec
22	22:44:02	SELECT status, COUNT(*) AS order_count FROM orders GROUP BY status ORDER BY order_count DE...	5 rows returned	0.000 sec / 0.000 sec
23	22:44:02	SELECT o.order_id, c.name AS customer_name, p.amount, p.method, p.status FROM orders o JO...	2 rows returned	0.000 sec / 0.000 sec
24	22:44:02	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 rows returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- mydatabase
- sakila
- shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_suppliers
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Administration Schemas Information

No object selected

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

121 ORDER BY order_count DESC, status;

122

123

124 -- Q9. Which active orders are still awaiting payment?

125 -- Non-cancelled orders whose payment is not yet 'Paid'.

126

127 SELECT

128 o.order_id,

129 c.name AS customer_name,

130 s.status AS order_status,

Result 21 Result 22 Result 23 Result 24 products 25 Result 26 Result 27 Result 28 Result 29 Result 30 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
28	13:08:31	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty ...	7 rows returned	0.000 sec / 0.000 sec
29	13:08:31	SELECT p.product_id, p.name, ROUND(AVG(r.rating), 2) AS average_rating, COUNT(r.review_id) A...	3 rows returned	0.000 sec / 0.000 sec
30	13:08:31	SELECT c.customer_id, c.name, c.email FROM customers c LEFT JOIN orders o ON c.customer_id ...	2 rows returned	0.000 sec / 0.000 sec
31	13:08:32	SELECT status, COUNT(*) AS order_count FROM orders GROUP BY status ORDER BY order_count DE...	5 rows returned	0.000 sec / 0.000 sec
32	13:08:32	SELECT o.order_id, c.name AS customer_name, o.status AS order_status, p.amount, p.method, ...	2 rows returned	0.000 sec / 0.000 sec
33	13:08:32	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 rows returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- mydatabase
- sakila
- shoposphere
 - addresses
 - categories
 - customers
 - order_items
 - orders
 - payments
 - product_suppliers
 - products
 - reviews
 - shipments
 - suppliers
 - views
 - Stored Procedures
 - Functions
 - sys
 - world

Administration Schemas Information

No object selected

Query 1 SQL File 3 SQL File 4 create Insert Sql customers queries

Limit to 1000 rows

133 JOIN payments p ON o.order_id = p.order_id

134 WHERE p.status = 'pending'

135 ORDER BY o.order_id;

136

137

138 -- Q10. Which suppliers provide products, and what is their supply price?

139

140 s.supplier_id,

141 s.name AS supplier_name,

Result 31 Result 32 Result 33 Result 34 products 35 Result 36 Result 37 Result 38 Result 39 Result 40 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
41	22:44:43	SELECT product_id, name, stock_qty FROM products WHERE stock_qty < 10 ORDER BY stock_qty ...	7 rows returned	0.000 sec / 0.000 sec
42	22:44:43	SELECT p.product_id, p.name, ROUND(AVG(r.rating), 2) AS average_rating, COUNT(r.review_id) A...	3 rows returned	0.000 sec / 0.000 sec
43	22:44:43	SELECT c.customer_id, c.name, c.email FROM customers c LEFT JOIN orders o ON c.customer_id ...	2 rows returned	0.000 sec / 0.000 sec
44	22:44:43	SELECT status, COUNT(*) AS order_count FROM orders GROUP BY status ORDER BY order_count DE...	5 rows returned	0.000 sec / 0.000 sec
45	22:44:43	SELECT o.order_id, c.name AS customer_name, p.amount, p.method, p.status FROM orders o JO...	2 rows returned	0.000 sec / 0.000 sec
46	22:44:43	SELECT s.supplier_id, s.name AS supplier_name, p.product_id, p.name AS product_name, ps.su...	10 rows returned	0.000 sec / 0.000 sec



9. Validation and Quality Checks

The dedicated normalization-validation script contains 14 checks.

#	Validation Check
1	Duplicate order items
2	Invalid customer references
3	Address/customer consistency
4	Orphan products in order items
5	Orphan orders in order items
6	Orphan categories
7	Orphan shipments
8	Orphan payments
9	Orphan reviews
10	Orphan product-supplier relationships
11	Duplicate shipments
12	Duplicate payments
13	Duplicate reviews
14	Duplicate product-supplier relationships

Expected clean-state result

Each validation query is designed to return zero rows when the corresponding integrity condition is satisfied. This provides a simple, repeatable database-quality check after data loading or modification.

Row-count validation

The insertion script includes queries that count rows in every table, calculate the total number of rows, compare the total against a target of 150, and return a PASS message when the total is at least 150.



10. Project Strengths

- Clear separation of business entities into dedicated relational tables.
- Primary keys and foreign keys are defined consistently.
- Composite keys are used appropriately in junction tables.
- Many-to-many product/supplier relationships are normalized through product_suppliers.
- Category hierarchy is represented using a self-referencing foreign key.
- CHECK constraints protect prices, stock, quantities, payments and ratings.
- Unique constraints prevent duplicate emails and duplicate customer/product reviews.
- Analytical queries cover products, customers, revenue, inventory, ratings, orders, payments and suppliers.
- Validation queries explicitly check for orphan records and duplicate relationships.
- The dataset is large enough to demonstrate joins and aggregation while remaining easy to inspect.

Key SQL concepts demonstrated

Concept	Examples
DDL	CREATE DATABASE, CREATE TABLE, DROP DATABASE
DML	INSERT INTO
DQL	SELECT
Joins	INNER JOIN, LEFT JOIN
Aggregation	SUM, AVG, COUNT, ROUND
Grouping	GROUP BY, HAVING
Filtering	WHERE
Sorting	ORDER BY
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK



11. Future Enhancements

Shopping cart. Add CARTS and CART_ITEMS to model items before checkout.

Multiple payment attempts. Remove the one-payment-per-order limitation if the application needs retries, partial payments or multiple transactions.

Discounts and coupons. Add product promotions and order-level coupon relationships.

Returns and refunds. Add return and refund entities to support post-purchase workflows.

Delivery tracking. Add tracking numbers and shipment status history.

Product media. Add a separate product-images table.

Advanced analytics. Add monthly revenue, repeat-customer, margin, delivery-time and inventory-turnover analysis.

Authentication. Add a dedicated account/authentication layer if the database becomes part of a production application.



12. Conclusion

Final project snapshot

Metric	Value
Database	ShopSphere
Implementation	MySQL 8+
Tables	11
Sample rows	202
Business questions	10
Validation checks	14
Primary-key strategy	Single and composite keys
Core relationship types	1:M, 1:1 and M:N
Normalization target	3NF-oriented design